Journey Service

You are tasked with implementing a user journey service that tracks users as they navigate through various stages. This is a rule-based framework where users onboard to journeys and progress through stages based on specific conditions.

A journey is a Directed Acyclic Graph (DAG) consisting of multiple stages. Users complete specific actions/events to advance from one stage to the next. Multiple users can onboard to the same journey, but each user can onboard to a journey only once.

Journeys can be of two types:

Time-bound: Has a start date and end date
Perpetual: No validity period, remains live until marked inactive
Journey stages:

First stage: Onboarding stage
Middle stages: Onward stages
Final stage: Terminal stage
By default, journeys are created as inactive. You must mark a journey as active for it to participate in evaluation. Time-bound journeys are automatically considered inactive when the current date falls outside their validity period (before start date or after end date).

Function Descriptions:

Implement the following methods in the JourneyService class:

createJourney(Journey journey): Creates a new journey in the system. The journey should be stored with isActive set to false by default. Returns the journey ID.
updateState(String journeyId, boolean active): Updates the active status of a journey. Returns true if the journey exists and was updated successfully, false if the journey doesn't exist.
getJourney(String journeyId): Retrieves and returns the Journey object for the given journey ID. Returns null if the journey doesn't exist.
evaluate(String userId, Payload payload): Evaluates a user action based on the payload. The payload contains an event type and attributes. This function should:
Check all active and valid journeys (perpetual journeys must be active; time-bound journeys must be active AND current date must be within start and end dates)
For matching, create a combined map of payload attributes + eventType, then check if all stage conditions exist in this combined map with matching values
If user is not onboarded and payload matches the first stage conditions: onboard the user to stage 1 of that journey
If user is already onboarded and payload matches the next stage conditions: move user to the next stage
A user can progress to multiple journeys in a single evaluate call
Returns a list of journey IDs where the user's status changed (onboarded or progressed). Return empty list if no changes occurred.
getCurrentStage(String userId, String journeyId): Returns the current stage number (1-indexed) of the user in the specified journey. Returns -1 if the user is not onboarded to the journey.

isOnboarded(String userId, String journeyId): Returns true if the user is part of the journey (has been onboarded), false otherwise.
Important Implementation Notes:

Current date for all evaluations: "2025-01-15"
Stage conditions must match exactly with the payload. To check matches, create a map that includes payload.attributes plus add "eventType" as a key with payload.eventType as value. Then verify all stage condition keys exist in this map with exact value matches.
For time-bound journeys, use string comparison for dates (format: "YYYY-MM-DD"). A journey is valid if: startDate <= currentDate <= endDate
Only active journeys participate in evaluation. Time-bound journeys must also be within their validity period.
When a user is at stage N and there are N+1 total stages, the user should match conditions of stages[N] (0-indexed array) to progress to stage N+1.
Users cannot skip stages - they must progress sequentially.
If multiple journeys match the same event, the user progresses in all matching journeys.
Classes Provided:

Journey: Contains journeyId (String), stages (List<Stage>), isActive (boolean), journeyType (String: "TIME_BOUND" or "PERPETUAL"), startDate (String or null), endDate (String or null)
Stage: Contains stageNumber (int, 1-indexed), conditions (Map<String, String>)
Payload: Contains userId (String), eventType (String), attributes (Map<String, String>)
Input Format:

The first line contains an integer Q, the number of queries.

Each query is on a separate line in one of the following formats:

CREATE <journeyId> <journeyType> <startDate> <endDate> <numStages> <stage1Conditions> <stage2Conditions> ...
UPDATE_STATE <journeyId> <active>
GET_JOURNEY <journeyId>
EVALUATE <userId> <eventType> <key1>:<value1> <key2>:<value2> ...
GET_CURRENT_STAGE <userId> <journeyId>
IS_ONBOARDED <userId> <journeyId>
Stage conditions format: key1=value1,key2=value2,...

Output Format:

For each query, output the result:

CREATE: Journey ID
UPDATE_STATE: "true" or "false"
GET_JOURNEY: Journey ID or "null"
EVALUATE: Comma-separated list of journey IDs where status changed, or "NONE" if no changes
GET_CURRENT_STAGE: Stage number (1-indexed) or "-1"
IS_ONBOARDED: "true" or "false"
Constraints:

$1 \leq Q \leq 1000$
$1 \leq$ Number of journeys ≤ 100
$1 \leq$ Number of stages per journey ≤ 20

1 ≤ Number of users ≤ 10^4
Journey IDs, user IDs, event types, and keys are non-empty strings with length ≤ 50
Values are non-empty strings with length ≤ 100
Dates are in "YYYY-MM-DD" format
Current date for evaluation: "2025-01-15"
Sample input

```
10
CREATE J1 PERPETUAL null null 3 eventType=LOGIN eventType=RECHARGE_PAGE eventType=RECHARGE_DONE
CREATE J2 TIME_BOUND 2025-01-01 2025-12-31 2 eventType=UPI_LITE_OPEN eventType=UPI_LITE_TOPUP
UPDATE_STATE J1 true
UPDATE_STATE J2 true
EVALUATE U1 LOGIN
EVALUATE U1 RECHARGE_PAGE
EVALUATE U2 LOGIN
EVALUATE U2 RECHARGE_PAGE
EVALUATE U2 RECHARGE_DONE
GET_CURRENT_STAGE U1 J1
```

Sample output

```
J1
J2
true
true
J1
J1
J2
J2
J2
2
```